

QRSMS: A SMS client with end-to-end encryption via Quick Response (QR) code

John Mel R. Ramos and Jaderick P. Pabico

I. INTRODUCTION

Despite the emergence of instant messaging (IM) and Over-the-top (OTT) messaging applications, short message service (SMS) remains as a popular option for mobile communication. With the first text message sent in 1992 through the Global System for Mobile communication (GSM) network, the technology has seen a lot of growth and development [1]. While mostly used for person-to-person communication [2], the medium has seen other applications such as alert notification [3], [4], empowering businesses and online services [5]–[7], mobile health (m-health) interventions [8], [9], mobile government applications [10], [11], education [12], Two-Factor Authentication (2FA) [13] and more. During the year 2022, Globe Telecom gained Php 8.8 billion revenue from SMS with 86.7 million mobile customers. Furthermore, the company has invested Php 1.1 billion to improve spam and fraudulent SMS detection and prevention [14]. Unreliable internet service as well as the lack of Internet infrastructure in some area remains a challenge in the adoption of online services for communication such as Facebook Messenger, Viber, Telegram, etc. SMS on the other hand is ubiquitous. Anyone with a mobile device and a registered subscriber identity module (SIM) card can send and receive text messages regardless of their platform and cellular provider [15].

Regardless of its wide range of applications and sustained use, however, SMS is not ideal for private communication such as sending sensitive and confidential information as early specifications for SMS does not have security in mind [16]. The European Telecommunications Standards Institute (ETSI) specified the use of A5 algorithm for encryption over the GSM network together with the A3 and A8 algorithm for authentication and cipher key generation to provide security [17] but attacks on the A5/1 and the weaker A5/2 algorithms has been demonstrated even with minimal resources [18]–[20] and even the A5/3 used for Third Generation (3G) network has been compromised [21]. Mobile network operators may also opt not to use any encryption during the transit of message wirelessly [22]. This means that a Man-in-the-Middle (MITM) can sit between the Mobile Equipment (ME) and Base Station Subsystem (BSS) using an International Mobile Subscriber Identity (IMSI) catcher to actively intercept traffic and even request the victim ME to use a weaker or no encryption at all [23]. Additionally, encryption is only performed between the Mobile Equipment (ME) and the Base Station Subsystem (BSS) [17]. Once the message reaches the SMS Center

(SMSC), messages are stored and kept in plain text for a while allowing malicious entities to target the SMSC to view the contents of these messages. This makes an end-to-end encryption a huge improvement over what is available in the GSM specifications.

To provide a better security system for SMS, independent researchers have proposed frameworks and protocols that offers end-to-end encryption. PK-SIM [24] provides end-to-end security between the mobile user and service provider through Public Key Infrastructure (PKI). SMSec [25] was developed using the Java Wireless Messaging API (WAP) and uses a combination of asymmetric and symmetric cryptography to provide security. EasySMS [26] and SecureSMS [7] are completely based on symmetric key cryptography. Similarly, SmartSMS [27] uses symmetric key encryption for m-health application. These protocols however, requires additional resources and infrastructure on the side of the service provider in order to be implemented.

This paper proposes an SMS client that uses quick-response (QR) codes for key exchange. Initially developed by Toyota's subsidiary Denso Wave for inventory of vehicle parts manufacturing, QR code has seen many applications in today's era [28]. Able to store up to a maximum of 7,089 characters in one code [29] it can be used to encode text, URL, and other data and can be decoded using a mobile device equipped with a camera and the appropriate software which can display a text, connect to a wireless network, open a webpage, or open a linked application [30]. The proposed client will encode the public key and parameters needed to generate a shared secret in a QR code. The code can be shared to the intended recipient through offline (in person) or other secure online platform that allows sending of images.

End-to-end security will be achieved through the combined use of symmetric and asymmetric cryptography as well a key-agreement protocol. Advanced Encryption Standard (AES) will be used for encrypting the message as it provides the most security while being performant [31]. Elliptic Curve Cryptography (ECC) will be used for generating key pairs and Elliptic Curve Diffie-Hellman (ECDH) for generating a shared secret. The use of ECC allows for a smaller key length while providing a comparable security as the Rivest–Shamir–Adleman (RSA) public key cryptosystem [32].

The proposed method will use the android platform to develop an SMS client that can generate key pair, exchange public keys through QR code, and establish an end-to-end encrypted communication.

II. LITERATURE REVIEW

Regardless of the insecurity present in SMS, due to its widespread application, it remains to be of use to many users around the world. To provide better end-to-end security, protocols/frameworks has been proposed.

PK-SIM card is a framework developed based on Public Key Infrastructure (PKI) providing end-to-end encryption between the Service Provider (SP) and the mobile user which contains the PK-SIM card. Their framework is composed of the mobile client, a Certification Authority and Registration Authority (CA/RA), a Secure Access gateway (SAG) found in the service provider's infrastructure and the Mobile Operator (MO) that provides the network for SMS communication. Before the user can send secure messages, the SAG together, with the CA/RA must first validate the identity of the mobile client. A primary key shared between the client and the SAG is generated during the authentication phase. This key is used to establish a session key which is used to symmetrically encrypt/decrypt the message. However, in their approach, end-to-end security is only available between the mobile client and the SAG on the Service Provider's side. Additionally, it is stated that the generation of primary key in the authentication phase takes up to a day or longer. Every time this key expires, the protocol must be performed again from the beginning [24]. Similarly, the SMSec protocol is designed such that it uses a combination of asymmetric cryptography for authenticating the user and uses symmetric cryptography for the actual exchange of messages. It also falls short such that end-to-end encryption is between the mobile device and the authentication source. Unlike PK-SIM however, SMSec is a software solution built with the Java Wireless Messaging API (WAP) [25].

In contrast, the EasySMS protocol is designed to provide end-to-end encryption using only symmetric algorithm. In their approach, an Authentication Server (AS) is responsible for the storage of all secret key shared between the AS and the respective Mobile Station (MS) or the user. It is also assumed that there is a shared secret key between the AS and a CA/RA which contains information about every mobile subscriber. Before a SIM card could be used, a subscriber must first register within the CA/RA. With these requirements met, the AS, with the help of CA/RA can now authenticate any MS that wants to send an SMS securely. However, this approach falls under the same disadvantage as the previous protocols. When user A wants to send a message to user B, the AS validates the identity of both users and generate different key for each respective user. This means that the message from user A is encrypted with the key shared between the AS and user A, decrypted in the AS then re-encrypt the message for user B using the key shared between the AS and user B [26]. This approach is also used for the development of SecureSMS but is proposed to be specific for value added services (VAS) and commercial applications [7]. SmartSMS follows a similar concept but instead of the AS being connected to a CA/RA, a trusted third party hosted in a public cloud is responsible for the functions of CA/RA [27].

In these protocols, apart from being not truly end-to-end

encrypted (encryption only occurs between the mobile station and the service provider), they also require a third party for authenticating the user. This means that in order to achieve secure communication, changes within the service provider's and the network itself must be considered. This paper will introduce a different approach that does not require additional infrastructure by having the cipher text sent through the SMS network while encryption and decryption is performed by the communicating devices.

III. OBJECTIVES

To create a secure platform for the exchange of encrypted SMS, the main objective of this research is to develop an SMS client application that can encrypt and decrypt messages and utilize QR codes as means of exchanging keys for generating a shared secret. To be specific, this research aims to do the following:

1. Design and implement a key-agreement protocol using QR code as means for key exchange.
2. Design and implement an SMS client for mobile android device with end-to-end encryption.
3. Conduct a usability study of the SMS client application.

IV. MATERIALS AND METHODS

A. Key-Agreement Protocol

To achieve end-to-end security, participants in the exchange should be able to generate a shared key known only to them. This process can be further divided into two steps. Keypair generation and shared secret key generation. This two step process is shown in Figure 1. The `javax.crypto` library contains the `KeyAgreement` class needed to establish a shared secret while the `java.security` library will enable us to use the `KeyGenerator` class which allows the generation of public and private key pair using EC. Since there are no built-in libraries for encoding and decoding of QR code, ZXing [33] and Google's ML kit bar code scanning [34] will be bundled with the application to provide the missing functionalities.

1) *Key pair Generation:* To be able to generate a shared secret with another user, each party involved must first generate their public and private key pair. For each contact the user wants to exchange keys with, a unique key pair will be generated using Elliptic Curve (EC) algorithm. The generated public key will be encoded to a QR code while the complementing private key will be kept securely in the system and will be used to finalize the key exchange.

2) *Shared Secret key Generation:* Once each party have their generated QR code, they will give their QR code to each other and scan the code to retrieve the key. Using ECDH, their private key for that particular user will be fed along with the scanned public key to generate a shared secret. This process is illustrated in fig. 1. The shared secret will be stored inside Android's built in KeyStore to protect the keys from unauthorized use [35].

It should be noted that in order to keep the integrity of the system, an assumption is made that the user will not feed a QR code whose origin is unknown. To ensure that the code being decoded is from the intended contact, the QR code

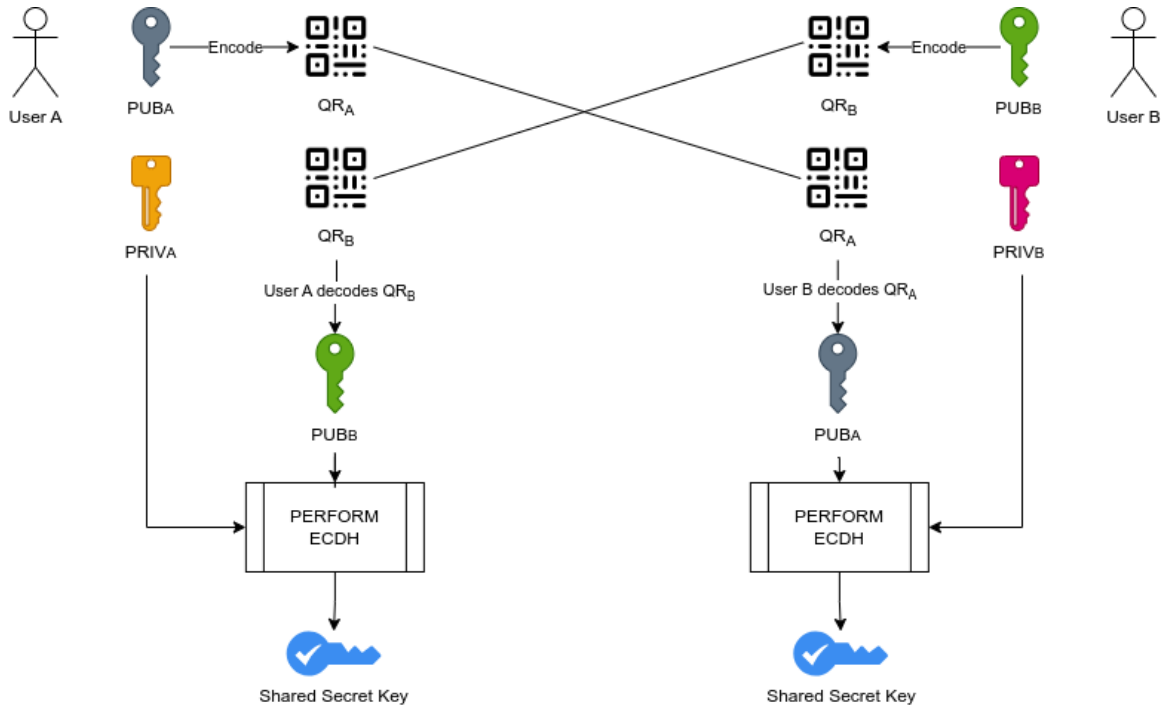


Fig. 1. Overview of key generation and key agreement in QRSMS

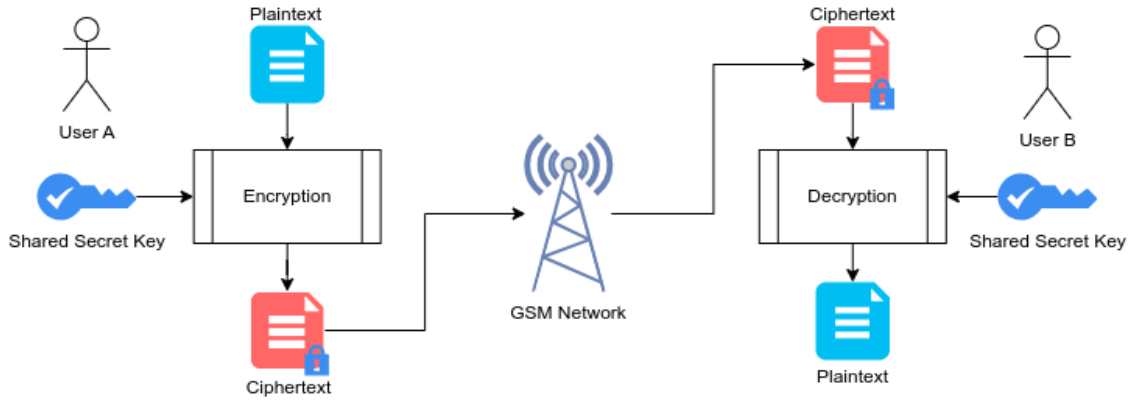


Fig. 2. Overview of sending and decryption of messages in QRSMS

should only come from the other party involved in the secure communication through other means such as offline (in-person exchange) or secure online application. This eliminates the need for authentication methods as the users themselves are the one responsible for ensuring that the participant in the exchange is the intended recipient.

B. Encrypted Messaging

To enable end-to-end encryption, both the sending and receiving party must have the application installed and have exchanged their QR code generated by the application through other means suggested below. In case only one party has the software installed, if they decide to send an encrypted message to a user without the application, or the recipient have not yet established the shared secret, the latter might receive the

message in cipher text, unable to read said text. If the opposite were to occur, the application should be able to determine if the received text was encrypted or not.

When each of the user involved in the key-agreement now having generated a shared secret with their intended contact, the exchange of encrypted SMS can now begin. AES encryption algorithm can be used generate the cipher text with the shared secret as the key for symmetric cryptography shown in fig. 2. The android platform comes with the `android.telephony` package contains the classes needed for sending and receiving SMS messages while the `javax.crypto` contains classes that allow the use of cryptographic libraries that will help achieve the goals of the application [36].

Equipped with features and support for developing android applications, Android Studio will be used as the integrated

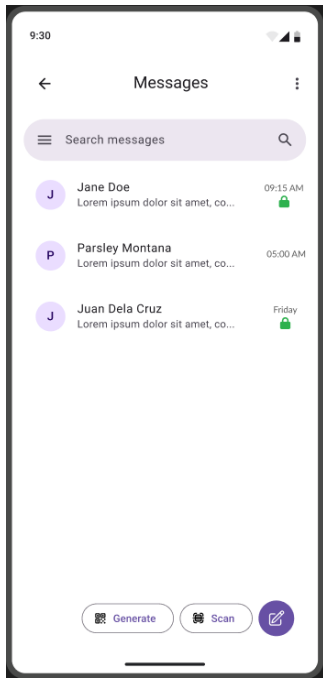


Fig. 3. QRSMS Inbox UI Prototype

development environment (IDE) to facilitate the development of the SMS client. The Kotlin programming language will also be used for the application as it fully supported for android development and it eliminates the verbosity of Java while allowing the use of Java libraries [37].

C. User Testing

Throughout the duration of the development process, the features and interface of the system will be thoroughly tested to ensure that the program can perform its intended functions. However, upon the completion of a minimum viable product, usability test will be conducted to further inspect the working condition of the application as well as analyze users' thoughts with regard to the system. Users will be asked to perform various tasks that involve the following capabilities of the application:

1. Generation of QR code containing the public key to be used for a contact
2. Scanning and decoding of QR code received from a contact
3. Sending of unencrypted and encrypted messages
4. Reading of unencrypted and encrypted messages
5. Management of stored keys

Along with the given tasks are questionnaire that aims to understand users' perspective throughout their usage of the application. Single Ease Question (SEQ) will be asked after completing each task. With measures of central tendency, a general sentiment can be derived on how well the users performed in each of the tasks. This can provide an insight on the ease-of-use of the system. This is followed by a System Usability Scale (SUS) Questionnaire to assess the system's perceived usability.

V. EXPECTED OUTCOME

A. Key-agreement with QR Code

Behind the SMS client's ability to conceal the contents of each message, the key-agreement protocol's function is vital to generate the secret key that will be used to encrypt each messages. Figure 4 shows the sequence of views that the user can see when they tap the generate QR in the inbox user interface (UI). On the other hand, when scanning a QR code, the sequence of views during this action is shown in figure 5. When users A and B wants to send each other encrypted messages, User A must first generate a QR code intended for user B and provide them the code and vice-versa. Through this exchange, one of the following cases may occur.

1) *Case 1: User A and B receives the correct code:* In this case, when user A generated a QR code they correctly supplied user B's phone number and gave them the correct QR code. User B does the same and upon scanning each other's given code, the key-agreement protocol is done successfully and came up with the same secret key for both user. Encrypted SMS can now occur.

2) *Case 2: User A or User B or Both receives the wrong code:* In this case, during generation or sending of QR code, either user A or user B selected the incorrect phone number or supplied the incorrect QR code. If this happened, the application should throw a user-friendly error message to allow them to troubleshoot the issue and take appropriate action.

B. SMS Client

Built for the android operating system, QRSMS is an SMS client that enables users to exchange end-to-end encrypted messages while allowing them to send regular SMS if they choose to. To ensure seamless operation by the user, the system should be designed with a simple and user-friendly interface such that users of any background can make operate the application without much difficulty. Figure 3 shows a prototype of the UI upon entering the application in which most of the significant functions of the system should be present and apparent. While users should be able to choose to send a regular or secure SMS, the system should be able to recognize if the message is encrypted or not and transform it if necessary to make it readable by the user.

In general, for the SMS client perform its intended function, the following requirements must be considered. The application should be able to:

1. Encrypt and send SMS message.
2. Decrypt and read SMS message.
3. Generate a private and public key pair then encode it into a QR Code.
4. Decode the QR code to generate the shared secret.
5. Allow users to manage saved keys.

C. Understanding System Usability and Ease of Use

In general, user testing allows for understanding how a system performs in real world applications. Having users test and evaluate the system enables the researchers to recognize

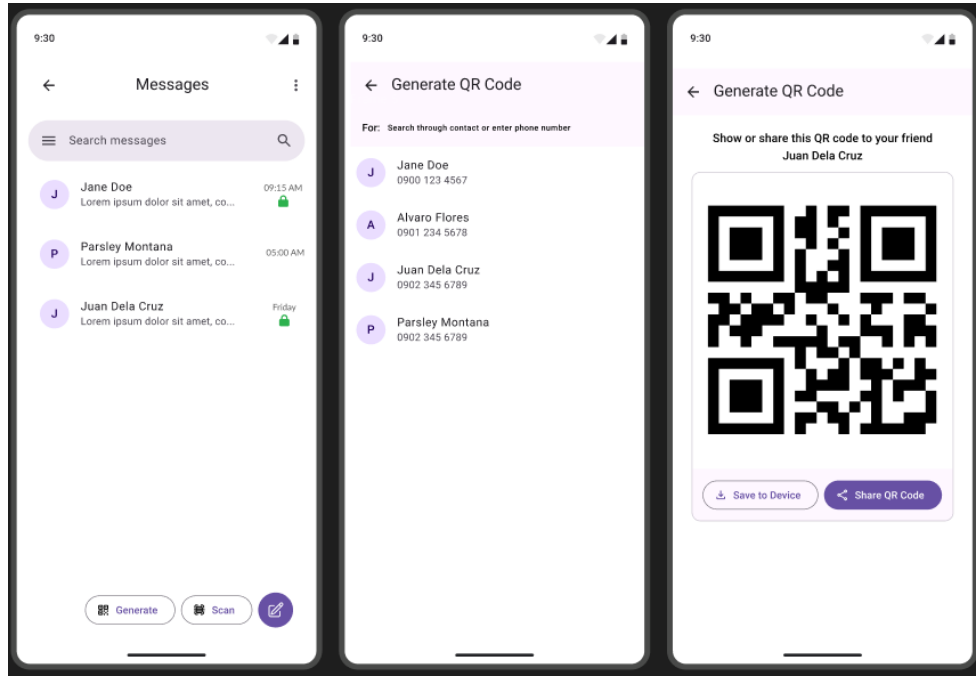


Fig. 4. QR Code generation view prototype

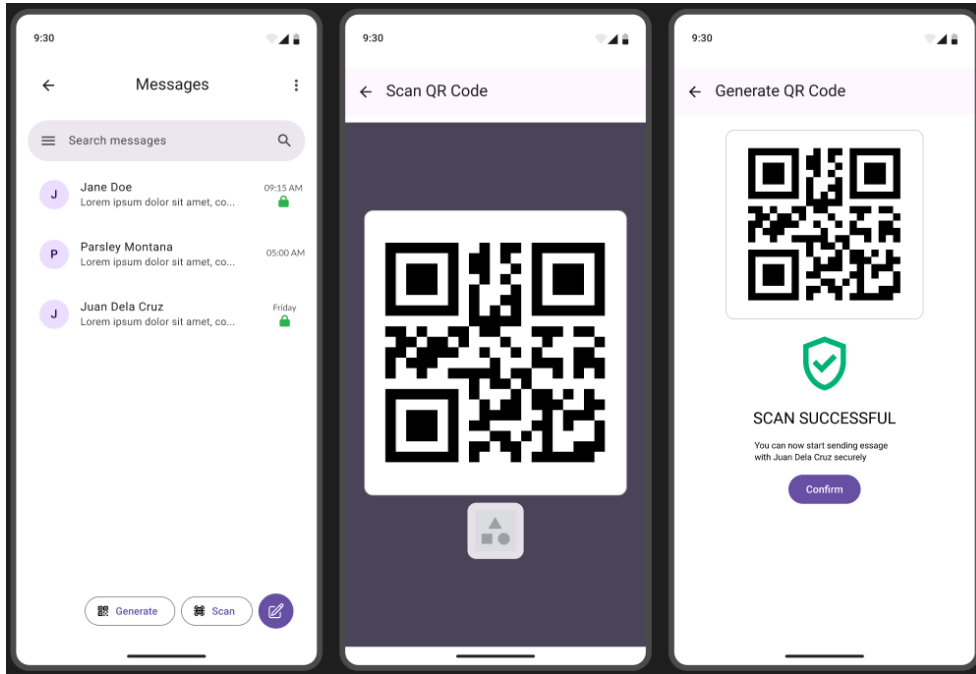


Fig. 5. QR Code scan view prototype

the potential of the proposed approach. With the way the key-agreement protocol is designed, users might initially find this method different from what they are used to. Users has to initiate the exchange themselves with the QR code to simplify the act of inputting of keys. But an intuitive and well-designed interface should be able to guide the users to take their intended actions. Because of the popularity of QR code, users generally recognizes it and knows how to use it. However, knowing what aspect of the system users found the most useful

and easy to operate can help identify further improvements to be made.

VI. IMPLEMENTATION

A. Key-agreement protocol with QR Codes

To allow users to exchange end-to-end encrypted messages, each participants in a conversation must first generate an elliptic curve (EC) keypair for their chosen contact. From the QRSMS app, this can be achieved by going from the inbox

screen, select "Generate QR", then select or input the contact that this keypair will be created for. This will then forward the user to a screen that shows a QR code that contains the public key from the generated keypair. They can show the QR code to the person, save it to their device or share it via other application (fig. 6). Once each of the participants have their partner's QR Code, they can start scanning the given code through their mobile phone's camera (fig. 7). The scanning process will check if the code is a valid code, and then attempt to initiate an Elliptic Curve Diffie-Hellman (ECDH) key agreement with the said contact. Once this step is done for both users, they can now begin exchanging encrypted messages.

1) *Key pair generation:* In order to generate an EC key pair for the given contact, an `EcKeyGen` class was created. This class uses the `KeyPairGenerator` class from the `java.security` package. Depending on the device's android version/ API level, generation of keypair can slightly differ. For devices with android version lower than 12 (API level 31 and below), an instance of this generator can be obtained using the `getInstance()` method providing the standard algorithm name that will be used for key pair generation, which in this case is "EC". The key pair generator can then be initialized using the `initialize()` method, passing an `ECGenParameterSpec` instance. In the case of QRSMS app, the `secp256r1` was used as the curve for the algorithm. The `generateKeyPair()` method can then be called which returns a public and private keypair. This keypair can then be saved in a Room database that is encrypted using `SQLCipher`.

On the otherhand, for devices of API level 31 and above, additional parameters can be provided in the methods above. To get an instance of the `KeyPairGenerator` class, an additional provider "AndroidKeyStore" can be passed to the `getInstance()` method to allow the generator to generate the key pair directly to Android's Keystore system. To initialize the generator, a parameter spec is still needed as its parameter. However, in this approach, the `KeyGenParameterSpec.Builder` class can be used to create the parameter spec. Simply supply the alias for the key entry and add the purpose of this key. The `KeyProperties` from the `android.security.keystore` package contains the `PURPOSE_AGREE_KEY` constant to ensure that this key can only be used for generating a shared secret using the ECDH algorithm. The builder class also provides methods such as `setAlgorithmParameterSpec()` where the `ECGenParameterSpec` can be passed so that the `AndroidKeyStore` generates an EC key pair. Once the `generateKeyPair()` method is called, a keypair entry will automatically be added in the keystore rather than having to manually add it in a database. The reason this can only be done for API level 31 and above is because the `PURPOSE_AGREE_KEY` constant was only added starting from this API.

2) *QR Code Generation:* Once a keypair is generated, the public key along with the phone number of the owner of that key is encoded in a QR code. Since the public key is a `PublicKey` object, it must first be decoded into byte array and then converted into a base64 string since the qr code generator encodes a string.

QR Code generation was achieved using the

`com.google.zxing` library adding it to the app as a dependency. With this library, the qr code generator can use the `qrcode.QRCodeWriter` class which contains the `encode()` method that can be used to generate the QR Code by passing the text to be encoded, the format/type of the Barcode, the width and height, and additional hints that can be added to the QR Code. A `QrUtils` object was created that provides the `generateQrCode()` method to handle this functionality. The encoded QR Code is converted as a `Bitmap` image so that it can be displayed to the user. The QR code can then be saved to the user's device or shared via other applications using android `Intents`.

3) *QR Code Scanning:* QR Code scanning capabilities are provided by android `CameraX` library as well as google's `MLKit Vision`. Google `MLKit Barcode Scanning` provides support for detecting different types of Barcode including QR Codes by providing the `BarcodeScannerOptions.Builder` class. The `CameraX` library enables the app to use the camera and attach its preview to a `PreviewView`. The `CameraX` package also has a library with built in integration of Google's `MLKit vision`. By using the `setImageAnalysisAnalyzer()` method, the `BarcodeScannerOptions` is passed to the `MLKitAnalyzer` class so that the camera can detect QR Codes and decode its content. If the analyzer detects a valid QR Code, it will then attempt to perform a key exchange.

4) *Key-Agreement:* The encoded text follows the following format - "normalized_phone_number_of_owner;public.key". After decoding the contents of the QR code, the application first checks if the contents is in the said format. If it is, the application proceeds to perform an ECDH key agreement.

The QRSMS source code includes an `EcdhKeyAgreement` class which takes the phone numbers of the involved parties, the public key in base64 format which is encoded in the given QR Code, and an instance of a `KeyManager` class which is responsible for managing all keypairs and secret keys within the application. This class uses the `KeyAgreement` class from the `javax.crypto` package that provides a method for generating a shared secret and an instance of the `KeyFactory` class from `java.security` package is used to reconstruct the public key material from a byte array. Once a shared secret has been derived, it is further processed through a Hash-Based Key Derivation function (HKDF) from `at.favre.lib.hkdf` to get a secret key that can be used for encryption and decryption. The involved users information such as their mobile phone as well as their public key information is added to the HKDF to the info parameter of the function to provide additional protection. The derived key is then stored inside the `AndroidKeyStore` for encryption and decryption.

B. SMS Client

QRSMS is a SMS application with support for end-to-end encryption for android devices. The application is available for android 7 (Nougat) up to Android 14. The client is built with the kotlin programming language and uses android `jetpack compose` for the UI framework.

While the generation and scanning of QR code is described in the previous sections, the client also provides users the

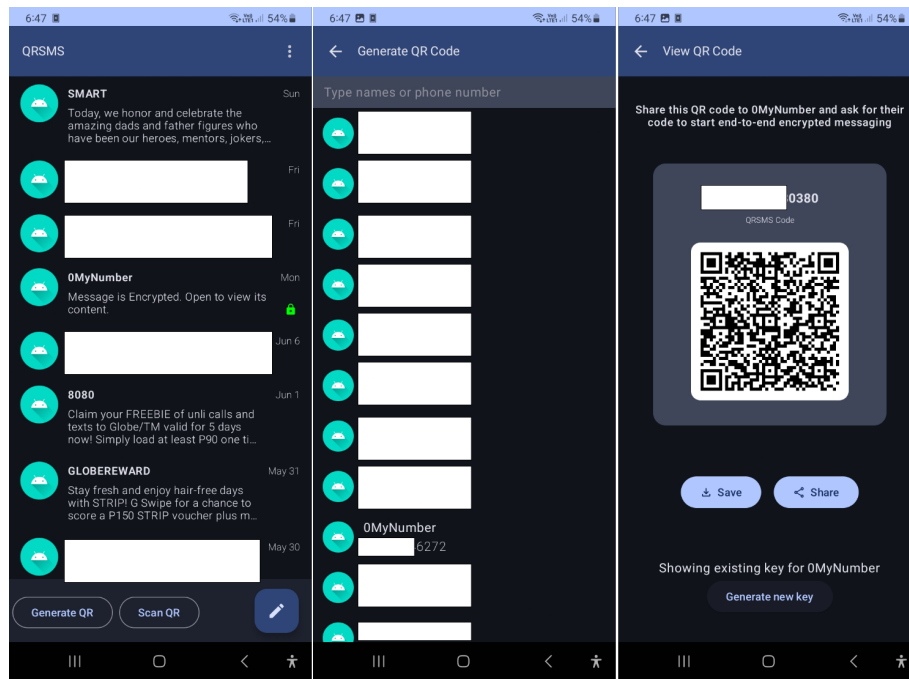


Fig. 6. QR Code Generation Process

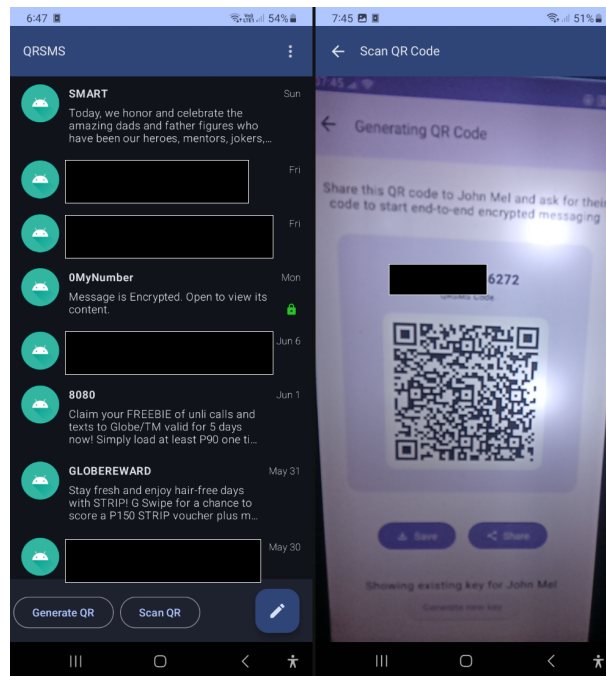


Fig. 7. QR Code Scanning Process

ability to send and read regular and encrypted messages. To send a message, the user can either select a conversation from the inbox screen which will take the user to the conversation screen that displays the thread of messages exchanged, or select the create message icon in the lower right portion of the screen. Using the create message approach moves the user to a screen where they can select a contact number or enter the phone number of the recipient.

In the conversation screen, if the user has an existing secret

key with the contact (e.g. they have previously performed a key-agreement), a select message type icon appears in front of the input box which will show a dialog where the user can select either to send a regular or an encrypted SMS message. Regardless of what type of message the user chooses to send, sending the the message is done through the SmsManager class that the android system provides. However, choosing to send an encrypted SMS message includes additional processing to the text before sending it to the recipient (fig. 8). On the other

hand, decryption of SMS message is done automatically by the application such that when the user opens a conversation that includes an encrypted text, the application will automatically decrypt the message if the key used to encrypt the message is available (fig. 9). If the same message is attempted to be viewed from the default sms app, users will only see a message in base 64 (fig. 10).

The app also provides a way to view and manage existing keys. The key management screen can be accessed by pressing the vertical ellipsis at the top right of the screen which brings out a menu where Manage Keys can be selected (fig. 11). Users can view the QR code which forwards them to the screen displayed after generating a QR Code and also a delete button. The screen also gives indication if end-to-end encryption is available for a given contact.

1) *Message Encryption*: If the user choose to send an encrypted message, the text is encrypted using the Cipher class from the javax.crypto package. Android supports the "AES/CBC/PKCS7Padding" transformation which is used to create an instance of the Cipher class. It is then initialized using the init() method providing the ENCRYPT_MODE constant to indicate that the cipher object is used for encryption as well as the secret key object obtained from the AndroidKeyStore. The doFinal() method performs the encryption process to the message and returns the ciphertext in byte array. The cipher object also automatically generates an initialization vector (IV) for each encryption. The IV is prepended before the cipher text and converted into base 64 string. Before sending the encrypted message to the receiver, the author chose the Start of Heading + Start of Text unicode characters to indicate the beginning of the encrypted message and the End of Text + End of Transmission characters to indicate the end. This characters are also converted to base 64 so that the SMS message will not be converted to UTF-8 encoding which reduces the maximum length of a message from 160 to just 70.

2) *Message Decryption*: Decryption of a message is done automatically when a conversation is opened. When rendering a message to the screen, the application checks if the current message is encrypted. If it is, processing is performed on the message to display the original text. Before initializing the cipher object, the IV must first be extracted from the base 64 string and then reconstructed using the IvParameterSpec from javax.crypto.spec package. With the IV extracted, the cipher object can now be initialized with DECRYPT_MODE as its mode of operation, the secret key shared with the contact, and the extracted iv. The doFinal() method can then be performed on the cipher text which will decrypt the given message.

REFERENCES

- [1] G. Le Bodic, *Mobile messaging technologies and services: SMS, EMS, and MMS*, 2nd ed. Chichester, West Sussex, England; Hoboken, NJ: J. Wiley & Sons, 2005, ch. 3, p. 47.
- [2] S. Herring, D. Stein, and T. Virtanen, *Pragmatics of Computer-Mediated Communication*. Walter de Gruyter, Jan. 2013, ch. 7, p. 162, google-Books-ID: 9cTmBQAAQBAJ.
- [3] C. W. Yoo, J. Lee, C. Yoo, and N. Xiao, "Coping behaviors in short message service (sms)-based disaster alert systems: From the lens of protection motivation theory as elaboration likelihood," *Information & Management*, vol. 58, no. 4, p. 103454, June 2021.
- [4] S. Azid, B. Sharma, K. Raghuwaiya, A. Chand, S. Prasad, and A. Jacquier, "Sms based flood monitoring and early warning system," *ARPJ Journal of Engineering and Applied Sciences*, vol. 10, no. 15, 2015.
- [5] D. Gargaro, "What is sms for business?" Sept. 2023. [Online]. Available: <https://www.business.com/articles/sms-for-business/>
- [6] S. Okazaki and C. R. Taylor, "What is sms advertising and why do multinationals adopt it? answers from an empirical study in european markets," *Journal of Business Research*, vol. 61, no. 1, p. 4–12, Jan. 2008.
- [7] N. Saxena and N. S. Chaudhari, "Securesms: A secure sms protocol for vas and other applications," *Journal of Systems and Software*, vol. 90, p. 138–150, Apr. 2014.
- [8] J. Tuckerman, K. Harper, T. R. Sullivan, A. R. Cuthbert, J. Fereday, J. Couper, N. Smith, A. Tai, A. Kelly, R. Couper, M. Friswell, L. Flood, C. C. Blyth, M. Danchin, and H. S. Marshall, "Short message service reminder nudge for parents and influenza vaccination uptake in children and adolescents with special risk medical conditions: The flutext-4u randomized clinical trial," *JAMA Pediatrics*, vol. 177, no. 4, p. 337–344, Apr. 2023.
- [9] S. L. King, J. Lebert, L. A. Karpisek, A. Phillips, T. Neal, and K. Kosyluk, "Characterizing user experiences with an sms text messaging-based mhealth intervention: Mixed methods study," *JMIR Formative Research*, vol. 6, no. 5, p. e35699, May 2022.
- [10] A. Onashoga, A. Ogunjobi, T. Ibharalu, and O. Lawal, "A secure framework for sms-based service delivery in m-government using a multicast encryption scheme," *African Journal of Science, Technology, Innovation and Development*, vol. 8, no. 3, p. 247–255, June 2016.
- [11] T. D. Susanto and R. Goodwin, "User acceptance of sms-based e-government services: Differences between adopters and non-adopters," *Government Information Quarterly*, vol. 30, no. 4, p. 486–497, Oct. 2013.
- [12] J. B. Hill, C. M. Hill, and D. Sherman, "Text messaging in an academic library: Integrating sms into digital reference," *The Reference Librarian*, vol. 47, no. 1, p. 17–29, July 2007.
- [13] A. Rhoda Iyanda and M. Ebenezer Fasasic, "Development of two-factor authentication login system using dynamic password with sms verification," *International Journal of Education and Management Engineering*, vol. 12, no. 3, p. 13–21, June 2022.
- [14] "Empowering customers with everyday digital solutions," Globe Telecom, Inc., Apr. 2023. [Online]. Available: <https://www.globe.com.ph/content/dam/globe/brie/AboutUs/investor-relations/integrated-report/Globe-2022-Integrated-Report.pdf>
- [15] E. F. Rosales, "Text messaging remains relevant in philippines," Nov. 2022. [Online]. Available: <https://www.philstar.com/business/2022/11/25/2226194/text-messaging-remains-relevant-philippines>
- [16] *Short Message Service (SMS) for Wideband Spread Spectrum Systems*, 3GPP2 Technical Requirement C.S0015-C, Nov. 2012. [Online]. Available: https://www.arib.or.jp/english/html/overview/doc/STD-T64v6.20/Specification/ARIB_STD-T64-C.S0015-Cv1.0.pdf
- [17] J. Schiller, *Mobile communications*, 2nd ed. London: Addison-Wesley, 2011, pp. 120–122.
- [18] A. Biryukov, A. Shamir, and D. Wagner, "Real time cryptanalysis of a5/1 on a pc," in *Fast Software Encryption*, ser. Lecture Notes in Computer Science, G. Goos, J. Hartmanis, J. van Leeuwen, and B. Schneier, Eds. Berlin, Heidelberg: Springer, 2001, p. 1–18.
- [19] A. Bogdanov, T. Eisenbarth, and A. Rupp, "A hardware-assisted realtime attack on a5/2 without precomputations," in *Cryptographic Hardware and Embedded Systems - CHES 2007*, ser. Lecture Notes in Computer Science, P. Paillier and I. Verbauwhede, Eds. Berlin, Heidelberg: Springer, 2007, p. 394–412.
- [20] E. Barkan, E. Biham, and N. Keller, "Instant ciphertext-only cryptanalysis of gsm encrypted communication," in *Advances in Cryptology - CRYPTO 2003*, ser. Lecture Notes in Computer Science, D. Boneh, Ed. Berlin, Heidelberg: Springer, 2003, p. 600–616.
- [21] O. Dunkelman, N. Keller, and A. Shamir, "A practical-time attack on the a5/3 cryptosystem used in third generation gsm telephony," *Cryptology ePrint Archive*, Paper 2010/013, 2010, publication info: Published elsewhere. Unknown where it was published. [Online]. Available: <https://eprint.iacr.org/2010/013>
- [22] *Digital cellular telecommunications system (Phase 2+); Security mechanisms for SIM application toolkit; Stage 2*, European Telecommunications Standards Institute Technical Specification 03.48, Dec. 2001. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/101100_101199/101181/08.08.00.60/ts_101181v080800p.pdf
- [23] G. Cattaneo, G. De Maio, P. Faruolo, and U. F. Petrillo, "A review of security attacks on the gsm standard," in *Information and Communication*

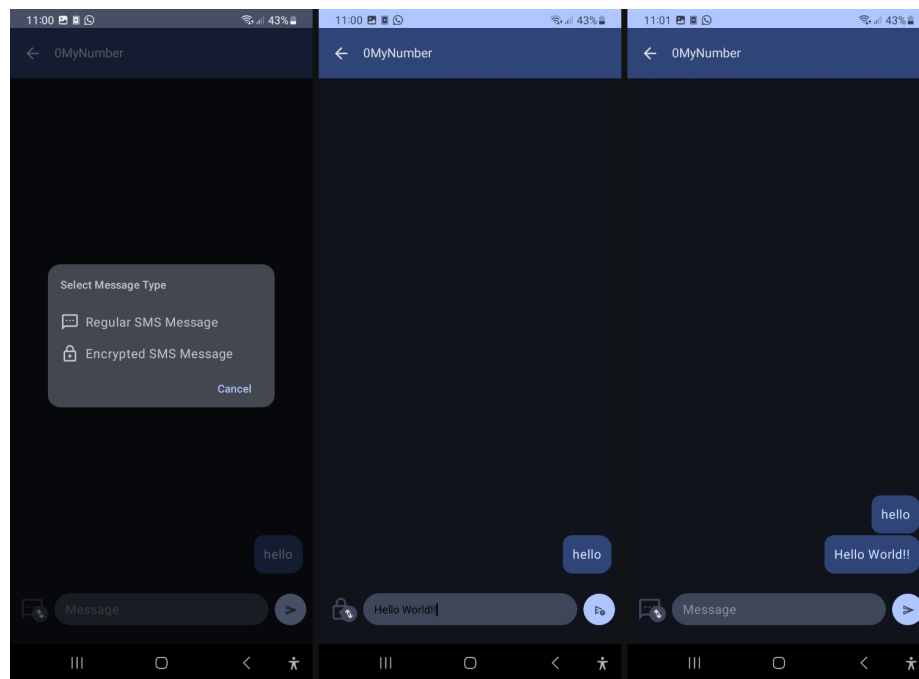


Fig. 8. Sending Encrypted Message

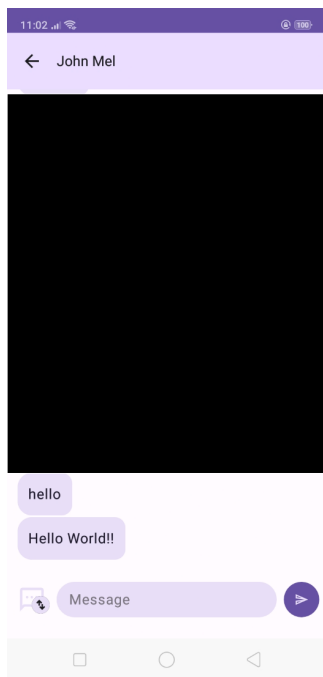


Fig. 9. Reading Encrypted Message

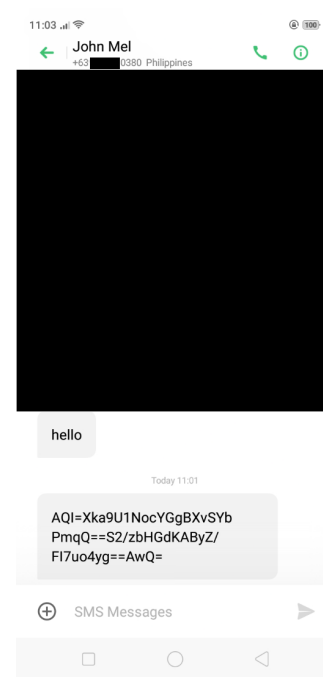


Fig. 10. Encrypted message as viewed in default sms app

Technology, ser. Lecture Notes in Computer Science, K. Mustofa, E. J. Neuhold, A. M. Tjoa, E. Weippl, and I. You, Eds. Berlin, Heidelberg: Springer, 2013, p. 507–512.

- [24] H. Rongyu, Z. Guolei, C. Chaowen, X. Hui, Q. Xi, and Q. Zheng, “A pk-sim card based end-to-end security framework for sms,” *Computer Standards & Interfaces*, vol. 31, pp. 629–641, June 2008.
- [25] J. L.-C. Lo, J. Bishop, and J. Eloff, “Smssec: An end-to-end protocol for secure sms,” *Computers & Security*, vol. 27, no. 5–6, p. 154–167, Oct. 2008. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0167404808000151>
- [26] N. Saxena and N. S. Chaudhari, “Easysms: A protocol for end-to-

end secure transmission of sms,” *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 7, p. 1157–1168, July 2014.

- [27] P. Vijayakumar, S. M. Ganesh, L. J. Deborah, and B. S. Rawal, “A new smartsms protocol for secure sms communication in m-health environment,” *Computers & Electrical Engineering*, vol. 65, p. 265–281, Jan. 2018.
- [28] S. Tiwari, “An introduction to qr code technology,” in *2016 International Conference on Information Technology (ICIT)*, Dec. 2016, p. 39–44. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7966807>
- [29] What is a qr code. [Online]. Available: <https://www.qrcode.com/en/>



Fig. 11. Key Management Screen

- about/
- [30] D.-H. Shin, J. Jung, and B.-H. Chang, "The psychology behind qr codes: User experience perspective," *Computers in Human Behavior*, vol. 28, no. 4, p. 1417–1426, July 2012.
 - [31] S. N. Karale, K. Pendke, and P. Dahiwal, "The survey of various techniques & algorithms for sms security," in *2015 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS)*, Mar. 2015, p. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7192943>
 - [32] M. Amara and A. Siad, "Elliptic curve cryptography and its applications," in *International Workshop on Systems, Signal Processing and their Applications, WOSSPA*, May 2011, p. 247–250. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5931464>
 - [33] zxing. [Online]. Available: <https://github.com/zxing/zxing>
 - [34] Scan barcodes with ml kit on android. [Online]. Available: <https://developers.google.com/ml-kit/vision/barcode-scanning/android>
 - [35] Android keystore system. [Online]. Available: <https://developer.android.com/privacy-and-security/keystore>
 - [36] Package index. [Online]. Available: <https://developer.android.com/reference/packages>
 - [37] M. Moskala and I. Wojda, *Android Development with Kotlin*. Packt Publishing Ltd, Aug. 2017, pp. 8–9, google-Books-ID: PJZGDwAAQBAJ.